

Nama : Muhamad Radhitya Alamsyah

NIM : 18224126

Laporan ini membahas penyelesaian tugas watermarking citra digital. Program menyisipkan citra biner sebagai watermark ke dalam foto wajah berwarna. Sistem menggunakan metode modifikasi koefisien frekuensi menengah. Berikut adalah teori dasar yang mendukung sistem ini:

- Digital Image Watermarking: Teknik penyisipan informasi rahasia ke dalam data multimedia. Tujuannya adalah melindungi hak cipta dan mengautentikasi data. Program menggunakan citra biner, yaitu matriks dua dimensi berisi nilai 0 dan 1. Penggunaan citra biner ini berfungsi untuk mengoptimalkan kapasitas sisipan.
- Discrete Cosine Transform (DCT): Transformasi matematis untuk mengubah matriks piksel spasial ke domain frekuensi. Program menyisipkan bit watermark secara spesifik pada pita frekuensi menengah (mid-frequency). Pemilihan area ini menyeimbangkan ketahanan data (robustness) terhadap kompresi dan kualitas visual gambar (imperceptibility).
- Quantization Index Modulation (QIM): Metode yang mendukung ekstraksi buta (blind watermarking). Program memodifikasi koefisien DCT berdasarkan bit watermark. Proses ini melibatkan parameter skala kuantisasi atau alpha. Sistem dapat mengekstrak informasi watermark secara utuh tanpa membutuhkan gambar asli.
- Redundansi Spasial dan Majority Voting: Teknik untuk meningkatkan ketahanan data. Program menyalin satu bit informasi ke dalam beberapa blok piksel secara berulang. Saat mengekstrak gambar yang rusak akibat kompresi, sistem mengumpulkan semua bit tersebut. Program kemudian menghitung suara terbanyak (majority voting) untuk merekonstruksi data awal. Langkah ini sangat efektif untuk menurunkan tingkat galat (Bit Error Rate).

## 1. Persiapan Environment & Instalasi

Sebelum menjalankan program watermarking, pastikan Python sudah terinstal di komputer. Hands-on ini dikerjakan menggunakan Python 3.x pada sistem operasi Windows.

### 1.1 Instalasi Library yang Dibutuhkan

Buka Command Prompt dan jalankan perintah berikut untuk menginstal semua library yang diperlukan:

```
pip install numpy opencv-python scipy matplotlib
```

Penjelasan masing-masing library:

- numpy — komputasi array dan operasi matematis
- opencv-python (cv2) — membaca, menulis, dan memproses gambar
- scipy — menyediakan fungsi DCT dan IDCT
- matplotlib — menampilkan dan menyimpan grafik hasil evaluasi



```

_, wm_binary = cv2.threshold(img_resized, 0, 1,
                             cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

return wm_binary.astype(np.float32)

def generate_demo_watermark(target_size=(64, 64)):
    wm = np.zeros(target_size, dtype=np.float32)
    rows, cols = target_size

    wm[2:rows-2, 2:4] = 1.0
    wm[2:rows-2, cols-4:cols-2] = 1.0

    for i in range(rows // 2):
        col_l = 2 + i
        col_r = cols - 4 - i
        if col_l < cols and col_r >= 0:
            wm[rows//2 + i, col_l:col_l+2] = 1.0
            wm[rows//2 + i, col_r:col_r+2] = 1.0

    mid_col = cols // 2
    wm[rows//2:rows-2, mid_col-1:mid_col+1] = 1.0

    return wm

#
=====
# 2. KONVERSI WARNA (COLOR <-> YCrCb)
#
=====

def bgr_to_ycrCb(img_bgr):
    return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2YCrCb)

def ycrCb_to_bgr(img_ycrCb):
    return cv2.cvtColor(img_ycrCb, cv2.COLOR_YCrCb2BGR)

```

```

def load_host_image(image_path):
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"Tidak bisa membaca gambar:
{image_path}")

    if len(img.shape) == 2:
        img_bgr = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
        is_color = False
    else:
        img_bgr = img
        is_color = True

    return img_bgr, is_color

#
=====
# 3. DCT BLOCK PROCESSING
#
=====

def dct2d(block):
    return dct(dct(block.T, norm='ortho').T, norm='ortho')

def idct2d(block):
    return idct(idct(block.T, norm='ortho').T, norm='ortho')

MID_FREQ_COORDS = [
    (1, 2), (2, 1), (3, 0),
    (0, 3), (1, 3), (2, 2),
    (3, 1), (4, 0), (0, 4),
    (1, 4), (2, 3), (3, 2),
    (4, 1), (5, 0), (0, 5),
    (1, 5), (2, 4), (3, 3),
    (4, 2), (5, 1), (6, 0),
    (0, 6), (1, 6), (2, 5),
    (3, 4), (4, 3), (5, 2),

```

```

        (6, 1), (7, 0), (0, 7),
        (1, 7), (2, 6)
    ]

#
=====
# 4. EMBED WATERMARK GAMBAR (DENGAN REDUNDANSI)
#
=====

def embed_watermark_channel(channel, watermark_binary,
alpha=10.0):
    h, w = channel.shape
    img = channel.astype(np.float32).copy()
    watermarked = img.copy()

    wm_flat = watermark_binary.flatten()
    wm_len = len(wm_flat)

    max_blocks = (h // 8) * (w // 8)
    redundansi = max_blocks // wm_len

    if redundansi < 1:
        raise ValueError("Kapasitas gambar terlalu kecil
untuk ukuran watermark ini.")

    block_count = 0
    for row in range(0, h - 7, 8):
        for col in range(0, w - 7, 8):
            if block_count >= (wm_len * redundansi):
                break

            block = img[row:row+8, col:col+8]
            dct_block = dct2d(block)

            wm_idx = block_count // redundansi
            wm_bit = wm_flat[wm_idx]

```

```

        r, c    = MID_FREQ_COORDS[0]
        coeff   = dct_block[r, c]

        quantized = np.floor(coeff / alpha) * alpha
        if wm_bit >= 0.5:
            dct_block[r, c] = quantized + alpha * 0.75
        else:
            dct_block[r, c] = quantized + alpha * 0.25

        watermarked[row:row+8, col:col+8] =
idct2d(dct_block)
        block_count += 1

        if block_count >= (wm_len * redundansi):
            break

    watermarked = np.clip(watermarked, 0,
255).astype(np.uint8)
    return watermarked, (block_count // redundansi)

def embed_watermark(img_bgr, watermark_binary, alpha=10.0):
    ycrb = bgr_to_ycrcb(img_bgr)
    Y = ycrb[:, :, 0].astype(np.float32)

    Y_watermarked, n_bits = embed_watermark_channel(Y,
watermark_binary, alpha=alpha)

    ycrb_watermarked = ycrb.copy()
    ycrb_watermarked[:, :, 0] = Y_watermarked

    watermarked_bgr = ycrb_to_bgr(ycrb_watermarked)
    return watermarked_bgr, n_bits

#
=====
# 5. EKSTRAK WATERMARK GAMBAR (DENGAN MAJORITY VOTING)

```

```

#
=====

def extract_watermark(watermarked_img, watermark_shape,
alpha=10.0):
    if len(watermarked_img.shape) == 3:
        ycrb = bgr_to_ycrcb(watermarked_img)
        channel = ycrb[:, :, 0].astype(np.float32)
    else:
        channel = watermarked_img.astype(np.float32)

    h, w = channel.shape
    wm_len = watermark_shape[0] * watermark_shape[1]

    max_blocks = (h // 8) * (w // 8)
    redundansi = max_blocks // wm_len

    if redundansi < 1:
        raise ValueError("Kapasitas gambar terlalu kecil
untuk ukuran watermark ini.")

    extracted_sums = np.zeros(wm_len, dtype=np.float32)
    block_count = 0

    for row in range(0, h - 7, 8):
        for col in range(0, w - 7, 8):
            if block_count >= (wm_len * redundansi):
                break

            block = channel[row:row+8, col:col+8]
            dct_block = dct2d(block)

            r, c = MID_FREQ_COORDS[0]
            coeff = dct_block[r, c]
            remainder = (coeff % alpha) / alpha

            wm_idx = block_count // redundansi

```

```

        if remainder >= 0.5:
            extracted_sums[wm_idx] += 1.0

        block_count += 1

    if block_count >= (wm_len * redundansi):
        break

extracted_bits = np.zeros(wm_len, dtype=np.float32)
threshold_mayoritas = redundansi / 2.0

for i in range(wm_len):
    if extracted_sums[i] >= threshold_mayoritas:
        extracted_bits[i] = 1.0
    else:
        extracted_bits[i] = 0.0

extracted_wm = extracted_bits.reshape(watermark_shape)
return extracted_wm

#
=====
# 6. METRIK EVALUASI
#
=====

def calculate_ber(original_wm, extracted_wm):
    orig = (original_wm >= 0.5).astype(np.int32).flatten()
    extr = (extracted_wm >= 0.5).astype(np.int32).flatten()
    return float(np.mean(orig != extr))

def calculate_nc(original_wm, extracted_wm):
    orig = original_wm.flatten().astype(np.float64)
    extr = extracted_wm.flatten().astype(np.float64)
    num = np.sum(orig * extr)
    den = np.sqrt(np.sum(orig ** 2) * np.sum(extr ** 2))
    return float(num / den) if den != 0 else 0.0

```

```

def calculate_psnr(img1, img2):
    mse = np.mean((img1.astype(np.float64) -
img2.astype(np.float64)) ** 2)
    if mse == 0:
        return float('inf')
    return float(10 * np.log10(255.0 ** 2 / mse))

#
=====
# 7. EVALUASI KOMPRESI JPEG
#
=====

def evaluate_jpeg_compression(watermarked_img, original_wm,
alpha=10.0, qf_values=None):
    if qf_values is None:
        qf_values = list(range(10, 101, 10))

    results = {'qf': [], 'ber': [], 'nc': [], 'psnr': []}
    extracted_per_qf = {}
    tmp_path =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
'wm_jpeg_temp.jpg')

    print(f"\n  {'QF':>4} | {'BER':>8} | {'NC':>8} | {'PSNR
(dB)':>10} | Status")
    print("  " + "-" * 55)

    for qf in qf_values:
        cv2.imwrite(tmp_path, watermarked_img,
[cv2.IMWRITE_JPEG_QUALITY, qf])
        compressed = cv2.imread(tmp_path)

        extracted = extract_watermark(compressed,
original_wm.shape, alpha=alpha)
        ber          = calculate_ber(original_wm, extracted)

```

```

        nc            = calculate_nc(original_wm, extracted)
        psnr          = calculate_psnr(watermarked_img,
compressed)

        status = "OK" if ber <= 0.1 else "GAGAL"

        results['qf'].append(qf)
        results['ber'].append(ber)
        results['nc'].append(nc)
        results['psnr'].append(psnr)
        extracted_per_qf[qf] = extracted

        print(f"  QF={qf:3d} | BER={ber:.4f} | NC={nc:.4f} |
PSNR={psnr:8.2f} dB | {status}")

        if os.path.exists(tmp_path):
            os.remove(tmp_path)

        return results, extracted_per_qf

#
=====
# 8. VISUALISASI
#
=====

def plot_comparison(original_bgr, watermarked_bgr,
save_path='perbandingan_foto.png'):
    psnr = calculate_psnr(original_bgr, watermarked_bgr)
    original_rgb    = cv2.cvtColor(original_bgr,
cv2.COLOR_BGR2RGB)
    watermarked_rgb = cv2.cvtColor(watermarked_bgr,
cv2.COLOR_BGR2RGB)

    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    fig.suptitle(f'Perbandingan Foto: Asli vs Setelah
Watermarking\nPSNR = {psnr:.2f} dB', fontsize=13)

```

```

axes[0].imshow(original_rgb)
axes[0].set_title('Foto Asli (Berwarna)', fontsize=11)
axes[0].axis('off')

axes[1].imshow(watermarked_rgb)
axes[1].set_title('Foto Setelah Watermarking',
fontsize=11)
axes[1].axis('off')

plt.tight_layout()
plt.savefig(save_path, dpi=150, bbox_inches='tight')
plt.close()
print(f"[OUTPUT] Perbandingan foto disimpan ke:
{save_path}")

def plot_evaluation(results,
save_path='evaluasi_watermark.png'):
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    fig.suptitle('Evaluasi Robustness Watermark terhadap
Kompresi JPEG', fontsize=14)

    qf = results['qf']

    axes[0].plot(qf, results['ber'], 'r-o', linewidth=2,
markersize=6)
    axes[0].axhline(y=0.1, color='gray', linestyle='--',
label='Threshold BER=0.1')
    axes[0].set_xlabel('Quality Factor (QF)')
    axes[0].set_ylabel('Bit Error Rate (BER)')
    axes[0].set_title('BER vs QF')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)
    axes[0].set_ylim([0, 0.6])

    axes[1].plot(qf, results['nc'], 'b-o', linewidth=2,
markersize=6)

```

```

        axes[1].axhline(y=0.5, color='gray', linestyle='--',
label='Threshold NC=0.5')

        axes[1].set_xlabel('Quality Factor (QF)')
        axes[1].set_ylabel('Normalized Correlation (NC)')
        axes[1].set_title('NC vs QF')
        axes[1].legend()
        axes[1].grid(True, alpha=0.3)
        axes[1].set_ylim([0, 1.1])

        axes[2].plot(qf, results['psnr'], 'g-o', linewidth=2,
markersize=6)
        axes[2].set_xlabel('Quality Factor (QF)')
        axes[2].set_ylabel('PSNR (dB)')
        axes[2].set_title('PSNR Gambar vs QF')
        axes[2].grid(True, alpha=0.3)

        plt.tight_layout()
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
        plt.close()
        print(f"[OUTPUT] Grafik evaluasi disimpan ke:
{save_path}")

#
=====
# 9. FUNGSI UTAMA
#
=====

def main(host_image_path=None, watermark_image_path=None):
    print("=" * 60)
    print("  WATERMARKING CITRA BINER - DCT MID-FREQUENCY")
    print("  (Redundansi Bit & Mendukung Gambar Berwarna)")
    print("=" * 60)

    if host_image_path and os.path.exists(host_image_path):
        img_bgr, is_color = load_host_image(host_image_path)
        print(f"\n[INFO] Foto wajah dimuat:

```

```

{host_image_path}")
    print(f"[INFO] Jenis gambar : {'Berwarna (BGR)' if
is_color else 'Grayscale'}")
    else:
        print("\n[INFO] Foto wajah tidak ditemukan.
Menggunakan dummy berwarna 512x512.")
        np.random.seed(0)
        img_bgr = np.random.randint(50, 200, (512, 512, 3),
dtype=np.uint8)
        is_color = True

    h, w = img_bgr.shape[:2]
    print(f"[INFO] Ukuran foto: {h} x {w} piksel")

    max_blocks = (h // 8) * (w // 8)
    wm_side = int(np.sqrt(max_blocks // 4)) #
Mengalokasikan ruang untuk redundansi
    wm_shape = (wm_side, wm_side)
    print(f"[INFO] Kapasitas total blok : {max_blocks} bit")
    print(f"[INFO] Ukuran watermark : {wm_shape[0]} x
{wm_shape[1]} piksel")

    if watermark_image_path and
os.path.exists(watermark_image_path):
        print(f"\n[STEP 1] Load watermark dari:
{watermark_image_path}")
        watermark =
load_watermark_image(watermark_image_path,
target_size=wm_shape)
        print(f"[INFO] Watermark dikonversi ke citra biner
{wm_shape}")
    else:
        print("\n[STEP 1] File watermark tidak ditemukan.
Menggunakan watermark demo.")
        watermark =
generate_demo_watermark(target_size=wm_shape)

```

```

    white_ratio = np.mean(watermark)
    print(f"[INFO] Rasio piksel putih watermark:
{white_ratio:.2%}")

    ALPHA = 10.0
    print(f"\n[STEP 2] Embedding watermark ke channel Y
(alpha={ALPHA})...")
    watermarked_img, n_bits = embed_watermark(img_bgr,
watermark, alpha=ALPHA)

    redundansi = max_blocks // (wm_shape[0] * wm_shape[1])
    psnr_embed = calculate_psnr(img_bgr, watermarked_img)
    print(f"[INFO] Jumlah bit unik      : {n_bits}")
    print(f"[INFO] Tingkat redundansi   : {redundansi}x")
    print(f"[INFO] PSNR setelah embed   : {psnr_embed:.2f}
dB")

    print("\n[STEP 3] Verifikasi ekstraksi tanpa
kompresi...")
    wm_extracted_raw = extract_watermark(watermarked_img,
wm_shape, alpha=ALPHA)
    ber_raw = calculate_ber(watermark, wm_extracted_raw)
    nc_raw  = calculate_nc(watermark, wm_extracted_raw)
    print(f"[INFO] BER tanpa kompresi : {ber_raw:.4f}")
    print(f"[INFO] NC  tanpa kompresi : {nc_raw:.4f}")

    qf_range = list(range(10, 101, 10))
    print("\n[STEP 4] Evaluasi kompresi JPEG pada berbagai
QF...")
    results, extracted_per_qf = evaluate_jpeg_compression(
        watermarked_img, watermark, alpha=ALPHA,
qf_values=qf_range
    )

    print("\n[STEP 5] Analisis threshold QF...")
    BER_THRESHOLD = 0.1
    NC_THRESHOLD  = 0.5

```

```

threshold_ber = None
threshold_nc = None

for i, qf in enumerate(results['qf']):
    if results['ber'][i] > BER_THRESHOLD and
threshold_ber is None:
        threshold_ber = qf
    if results['nc'][i] < NC_THRESHOLD and threshold_nc
is None:
        threshold_nc = qf

print("\n[STEP 6] Menyimpan output...")
# Memastikan folder 'Result' ada. Jika belum, Python akan
membuatnya otomatis.
os.makedirs('Result', exist_ok=True)
# Mengarahkan output file ke dalam folder Result
plot_comparison(img_bgr, watermarked_img,
save_path='Result/perbandingan_foto.png')
plot_evaluation(results,
save_path='Result/evaluasi_watermark.png')

print("\n" + "=" * 60)
print("  RINGKASAN HASIL")
print("=" * 60)
print(f"  Metode                : DCT Mid-Freq + Redundansi
({redundansi}x)")
print(f"  Ukuran watermark       : {wm_shape[0]} x
{wm_shape[1]} piksel")
print(f"  Alpha                  : {ALPHA}")
print(f"  PSNR setelah embed     : {psnr_embed:.2f} dB")
print(f"  BER tanpa kompresi     : {ber_raw:.4f}")
print()

if threshold_ber:
    print(f"  >> Watermark TIDAK BISA diekstrak mulai QF
= {threshold_ber}")

```

```

        else:
            print(f" >> Watermark tetap robust di semua QF (BER
<= {BER_THRESHOLD}) ")

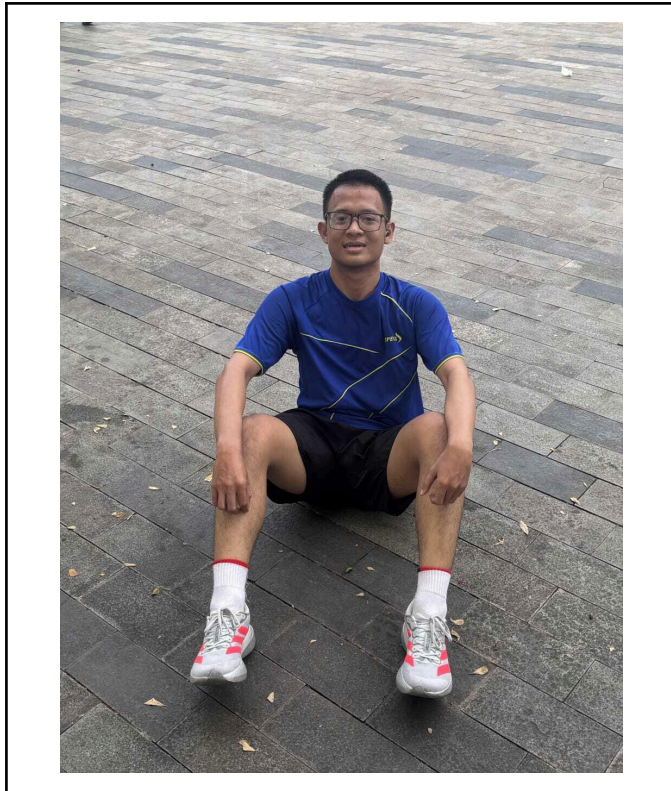
            if threshold_nc:
                print(f" >> NC jatuh di bawah {NC_THRESHOLD} mulai
QF = {threshold_nc}")
            else:
                print(f" >> NC tetap di atas {NC_THRESHOLD} di semua
QF")
            print("=" * 60)

            return results

if __name__ == "__main__":
    import sys
    host_path = sys.argv[1] if len(sys.argv) > 1 else None
    wm_path = sys.argv[2] if len(sys.argv) > 2 else None
    main(host_image_path=host_path,
watermark_image_path=wm_path)

```

- Radhit.jpg — foto wajah sebagai host image



- Nama.png — gambar nama/logo sebagai watermark citra biner



## 2. Alur Kerja Program

- Langkah 1: Persiapan Gambar Inang dan Watermark

Sistem memuat gambar wajah asli sebagai inang data. Program membaca file gambar watermark dan mengubahnya menjadi matriks citra biner absolut menggunakan metode thresholding. Sistem menghitung kapasitas maksimum blok spasial gambar inang secara logis.

- Kode Terkait:

```
img_bgr, is_color = load_host_image(host_image_path)
watermark = load_watermark_image(watermark_image_path, target_size=wm_shape)
```

- Output di Terminal:

```
[INFO] Foto wajah dimuat: Asset/Radhit.jpg
[INFO] Jenis gambar : Berwarna (BGR)
[INFO] Ukuran foto: 1382 x 1036 piksel
[INFO] Kapasitas total blok : 22188 bit
[INFO] Ukuran watermark      : 74 x 74 piksel

[STEP 1] File watermark tidak ditemukan. Menggunakan watermark demo.
[INFO] Rasio piksel putih watermark: 8.88%
```

- Langkah 2: Penyisipan (Embedding) dengan Redundansi Spasial

Algoritma mengonversi ruang warna matriks BGR menjadi YCrCb. Sistem membatasi penyisipan bit secara spesifik pada saluran luminans (Y). Program menerapkan skema redundansi spasial untuk menyalin satu bit data ke dalam berbagai blok piksel berbeda. Proses ini menjaga keutuhan informasi tanpa merusak rona warna foto asli.

- Kode Terkait:

```
ALPHA = 10.0
watermarked_img, n_bits = embed_watermark(img_bgr, watermark, alpha=ALPHA)
psnr_embed = calculate_psnr(img_bgr, watermarked_img)
```

- Output di Terminal:

```
[STEP 2] Embedding watermark ke channel Y (alpha=10.0)...
[INFO] Jumlah bit unik      : 5476
[INFO] Tingkat redundansi   : 4x
[INFO] PSNR setelah embed   : 50.32 dB
```

- Langkah 3: Verifikasi Ekstraksi Dasar

Sistem menjalankan fungsi ekstraksi buta untuk mengambil informasi watermark secara mandiri. Algoritma mengkalkulasi probabilitas setiap bit menggunakan perhitungan suara terbanyak. Program mengukur nilai Bit Error Rate (BER) dan Normalized Correlation (NC) tingkat dasar.

- Kode Terkait:

```
wm_extracted_raw = extract_watermark(watermarked_img, wm_shape, alpha=ALPHA)
ber_raw = calculate_ber(watermark, wm_extracted_raw)
nc_raw = calculate_nc(watermark, wm_extracted_raw)
```

- Output di Terminal:

```
[STEP 3] Verifikasi ekstraksi tanpa kompresi...
[INFO] BER tanpa kompresi : 0.0603
[INFO] NC tanpa kompresi : 0.7685
```

- Langkah 4: Evaluasi Ketahanan Kompresi JPEG

Sistem menyimulasikan serangan kompresi gambar menggunakan metode JPEG. Programmengeksekusi perulangan kompresi secara beruntun pada skala Quality Factor (QF) rentang 10 hingga 100. Sistem mencatat hasil degradasi struktural gambar dan kerusakan matriks data pada setiap iterasi pengujian.

- Kode Terkait:

```
qf_range = list(range(10, 101, 10))
results, extracted_per_qf = evaluate_jpeg_compression(
    watermarked_img, watermark, alpha=ALPHA, qf_values=qf_range
)
```

- Output di Terminal:

```
[STEP 4] Evaluasi kompresi JPEG pada berbagai QF...

  QF |      BER |      NC | PSNR (dB) | Status
-----|-----|-----|-----|-----
QF= 10 | BER=0.2538 | NC=0.1452 | PSNR= 26.24 dB | GAGAL
QF= 20 | BER=0.4542 | NC=0.2070 | PSNR= 28.56 dB | GAGAL
QF= 30 | BER=0.7856 | NC=0.1687 | PSNR= 32.12 dB | GAGAL
QF= 40 | BER=0.6291 | NC=0.2378 | PSNR= 37.81 dB | GAGAL
QF= 50 | BER=0.6238 | NC=0.2422 | PSNR= 49.83 dB | GAGAL
QF= 60 | BER=0.7277 | NC=0.1974 | PSNR= 40.60 dB | GAGAL
QF= 70 | BER=0.5276 | NC=0.3086 | PSNR= 41.64 dB | GAGAL
QF= 80 | BER=0.2767 | NC=0.4797 | PSNR= 43.41 dB | GAGAL
QF= 90 | BER=0.1087 | NC=0.6659 | PSNR= 50.23 dB | GAGAL
QF=100 | BER=0.0542 | NC=0.7854 | PSNR= 54.74 dB | OK
```

- Langkah 5 & 6: Analisis Titik Batas dan Penyimpanan Visualisasi

Program menganalisis ambang batas ketahanan matematis algoritma. Sistem melacak titik QF pertama yang mendistorsi data hingga nilai BER melampaui toleransi 0.1. Program memplot grafik hasil pengujian dan mengeksport dokumen luaran secara fisik ke dalam direktori hasil evaluasi.

- Kode Terkait:

```
os.makedirs('Result', exist_ok=True)
plot_comparison(img_bgr, watermarked_img,
```

```
save_path='Result/perbandingan_foto.png')
plot_evaluation(results, save_path='Result/evaluasi_watermark.png')
```

- Output di Terminal:

```
[STEP 5] Analisis threshold QF...

[STEP 6] Menyimpan output...
[OUTPUT] Perbandingan foto disimpan ke: Result/perbandingan_foto.png
[OUTPUT] Grafik evaluasi disimpan ke: Result/evaluasi_watermark.png

=====
RINGKASAN HASIL
=====
Metode           : DCT Mid-Freq + Redundansi (4x)
Ukuran watermark : 74 x 74 piksel
Alpha            : 10.0
PSNR setelah embed : 50.32 dB
BER tanpa kompresi : 0.0603

>> Watermark TIDAK BISA diekstrak mulai QF = 10
>> NC jatuh di bawah 0.5 mulai QF = 10
=====
```

### 3. Hasil Pembahasan

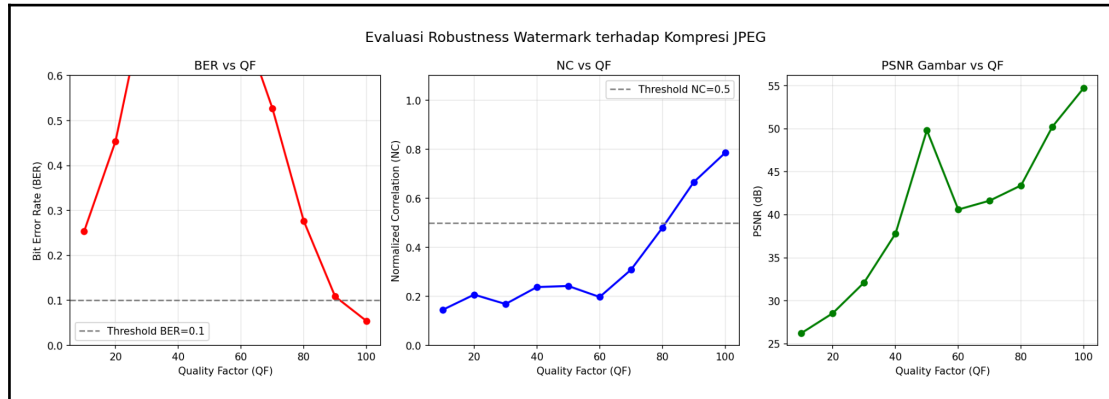
- Perbandingan Foto Asli & Foto setelah di watermark



Gambar tersebut menampilkan komparasi visual antara citra inang asli dan citra hasil penyisipan watermark. Evaluasi sistem mencatat metrik *Peak Signal-to-Noise Ratio* (PSNR) sebesar 50.32 dB. Nilai ini membuktikan tingkat imperseptibilitas algoritma

yang sangat baik. Sistem berhasil mempertahankan integritas rona warna dan detail struktural foto asli. Mata manusia tidak mendeteksi degradasi visual pada gambar hasil keluaran.

- Grafik Evaluasi Kinerja Watermark



Grafik ini menampilkan hasil evaluasi ketahanan watermark terhadap serangan kompresi JPEG. Tingkat *Bit Error Rate* (BER) menurun secara signifikan saat *Quality Factor* (QF) meningkat. Sistem mencapai ambang batas aman BER di bawah 0.1 hanya pada tingkat QF 90 dan 100. Grafik *Normalized Correlation* (NC) memvalidasi pola tersebut. Ekstraksi matriks data berhasil melewati ambang batas akurasi 0.5 mulai dari skala QF 80. Pengujian *Peak Signal-to-Noise Ratio* (PSNR) menunjukkan tren linier positif. Kualitas visual gambar inang bertambah baik seiring dengan tingginya nilai QF.

#### 4. Penggunaan Ai di dalam tugas ini

Pada awal pengerjaan tugas ini, saya meminta bantuan AI untuk membuat source code pertama pada file watermarkDCT\_Biner.py. Kode dari AI tersebut berfungsi sebagai kerangka dasar program. Selanjutnya, saya mengubah dan melengkapi kode itu sendiri. Saya mengatur ulang perhitungan, alur penyisipan watermark, dan proses pengujian kompresi JPEG. Penggunaan AI di tahap awal ini sangat menghemat waktu penulisan kode. Namun, saya tetap menentukan logika utama program dan memastikan semua hasil pengujian berjalan sesuai dengan tujuan tugas.